

Programovací jazyky a překladače

Přednáška 4, poznámky

Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

Témata

- **ADMINISTRATIVA:** 20.3. bude zveřejněno zadání semestrálního projektu
- Způsoby návrhu, překladu a vykonávání programovacích jazyků
 - HW a virtuální stroj
 - Zásobníkové a registrové procesory
 - Překladače a interprety
 - JIT, AOT: Java, Microsoft, Android
 - T diagramy
- Formální jazyky a jejich generující a akceptující struktury
 - Gramatiky a automaty, Chomského hierarchie
 - Vlastnosti gramatik
 - Pravidla, derivace a stromy
 - Víceznačnost
 - Arita, asociativita, priorita/precedence
 - Konečné automaty, úvodní informace
- Vzorové příklady zkouškového typu (bude-li čas)
- Prezentace z oblasti LL gramatik (témata souvisí s následující přednáškou)

Prezentace

- LL gramatiky, rozdělení a vlastnosti
- Syntaktická analýza shora dolů
- Syntaktická analýza rekurzivním sestupem

- %s <STAV> : zahrnuté stavy (platí všechny včetně explicitně neoznačených)
 - %x <STAV> : vyloučené stavy (platí pouze stavy, označené návěštím)
-

```
%x comment
```

```
%%
```

```
int line_num = 1;
```

```
"/*"
```

```
BEGIN(comment);
```

```
<comment>[ ^* ]*
```

```
/* ignoruj cokoli mimo * */
```

```
<comment>"*" + [ ^* /\ ]*
```

```
/* ignoruj *, nenásledované / */
```

```
<comment>\n
```

```
++line_num;
```

```
<comment>"*" + "/"
```

```
BEGIN(INITIAL);
```

%X S1, S2, S3

%%

"/"

<S1>"*"

<S2>[^*]

<S2>"*"

<S3>"*"

<S3>[^*/]

<S3>"/"

BEGIN(S1);

BEGIN(S2);

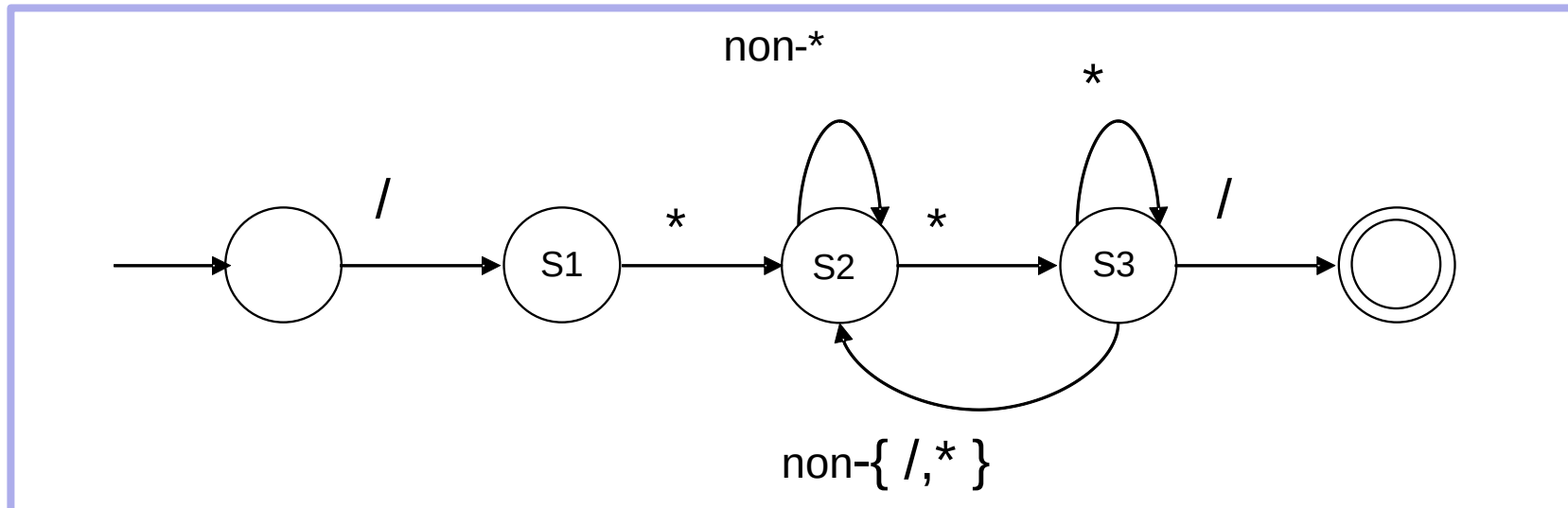
; /* zůstaň v S2 */

BEGIN(S3);

; /* zůstaň v S3 */

BEGIN(S2);

BEGIN(INITIAL);



Lexikální analýza

Miroslav Beneš
Dušan Kolář



Konstruktory překladačů

Miroslav Beneš
Dušan Kolář



Překladače a interprety, charakteristika

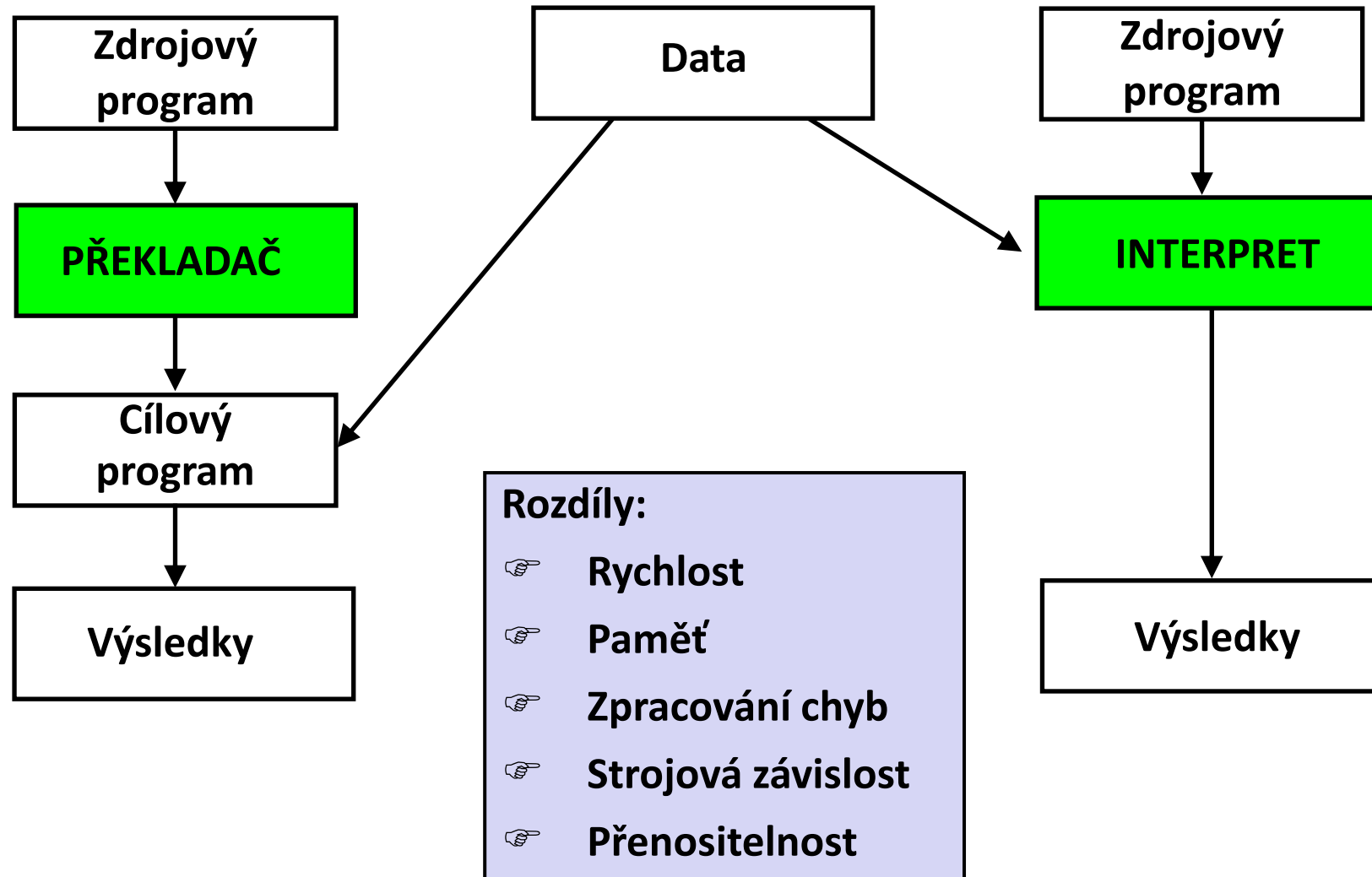
Překladač

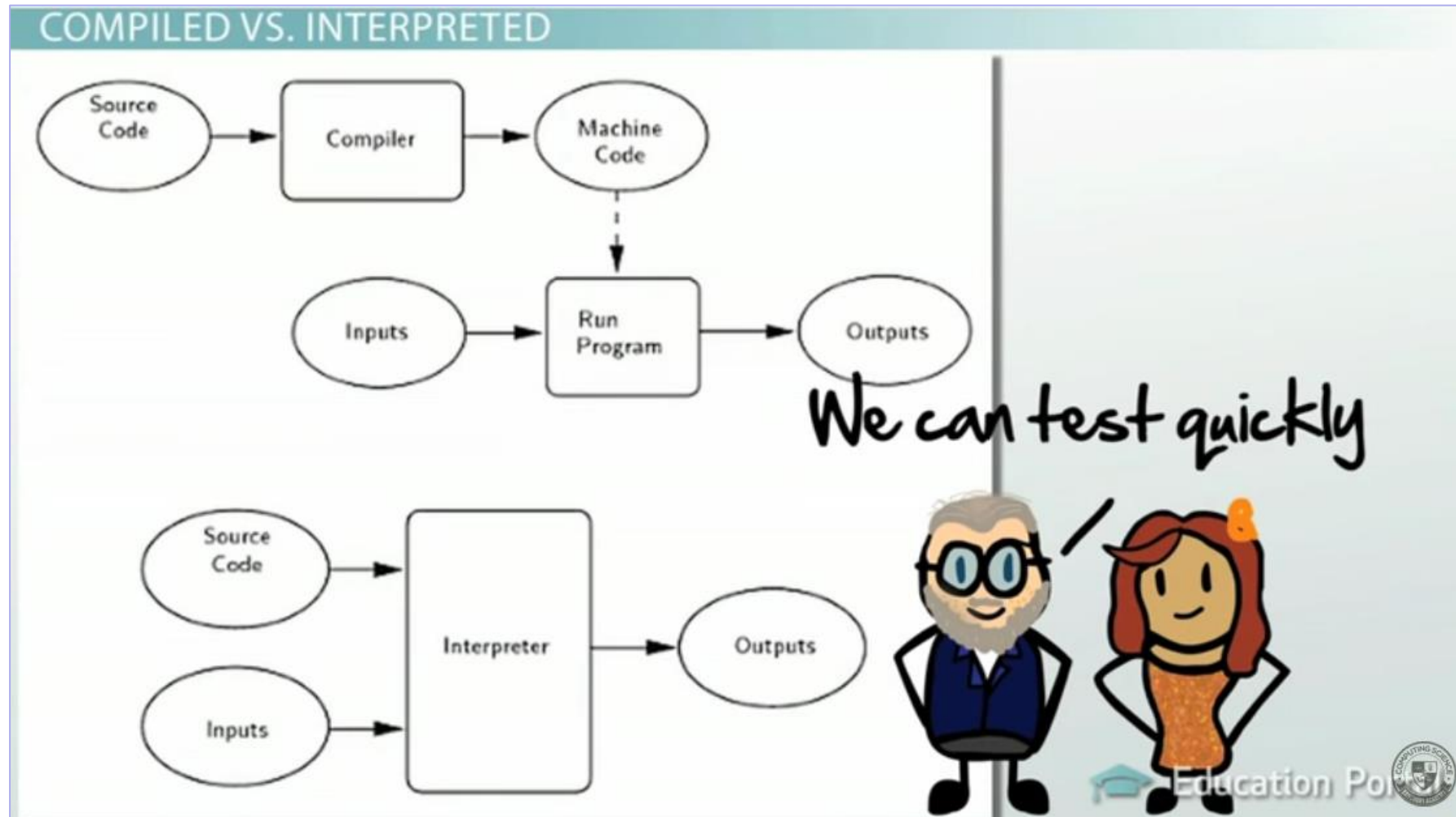
- Pracuje s celými soubory
- Z kódu ve zdrojovém jazyce vytvoří program ve strojovém kódu
- Výsledný program je na daném HW spustitelný jako celek

Interpret

- Pracuje s jednotlivými příkazy nebo výrazy, které okamžitě vykonává pomocí různých technik (viz následující slajd)
- Nevytváří spustitelný soubor a nebývá vázán na konkrétní HW. Jeho implementační platformou jsou spíše přenositelné virtuální stroje, ale to neznamená, že neběží i na běžných procesorech

Tradiční překladač a interpret, srovnání

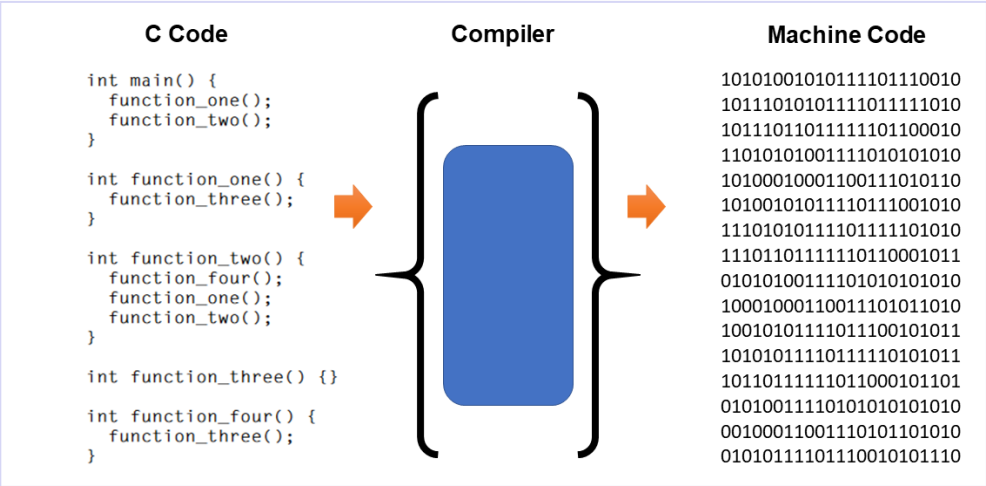




Interpretační techniky

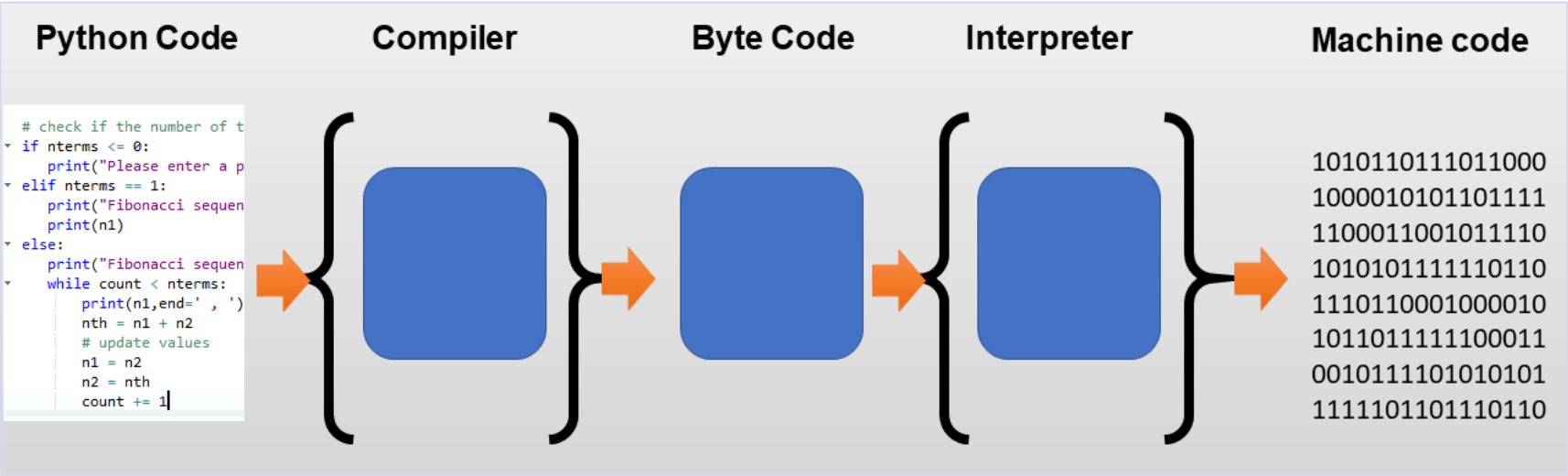
- Části interpretovaného kódu jsou kompilovány do mezikódu (např. Java bytecode) a následně mohou být:
 - Výhradně krok za krokem interpretovány příslušnou instrukční sadou cílového procesoru nebo virtuálního stroje (ISCS)
 - Části (byte)kódu, opakující se vícekrát než je zvolený práh nebo identifikované pomocí profileru (hot-spots), jsou během interpretace kompilovány do ISCS a následně vykonávány místo jednotlivých instrukcí
 - **JIT** (Just In Time, On-The-Fly), lepší možnosti optimalizace, vyšší časové i paměťové nároky, horší přenositelnost
 - Celý mezikód je překompilován ještě před prvním spuštěním
 - **AOT** (Ahead Of Time)

Současný překladač a interpret, srovnání

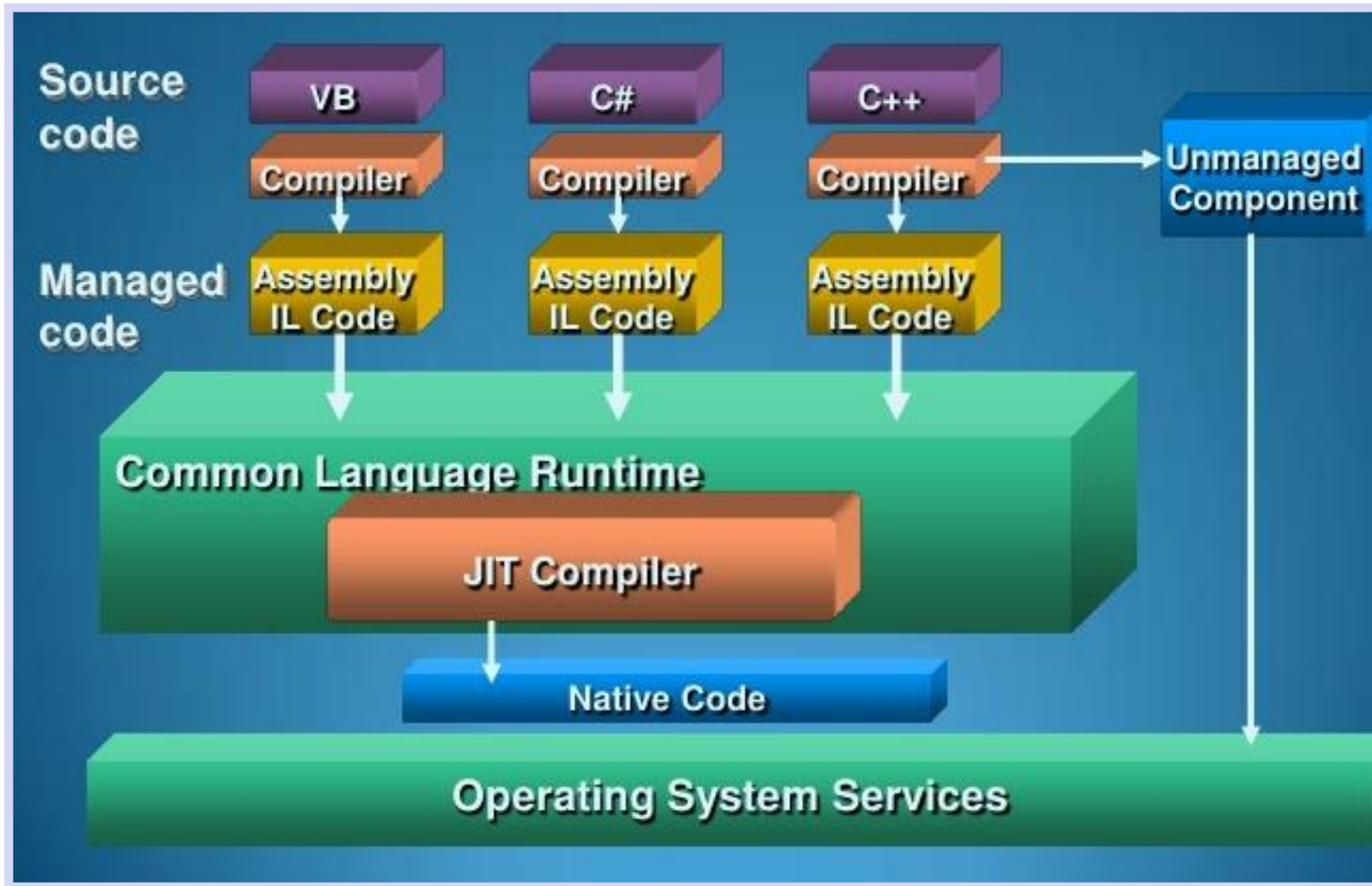


Compiler:

- Scan
- Parse
- Kontext
- Obecný mezikód



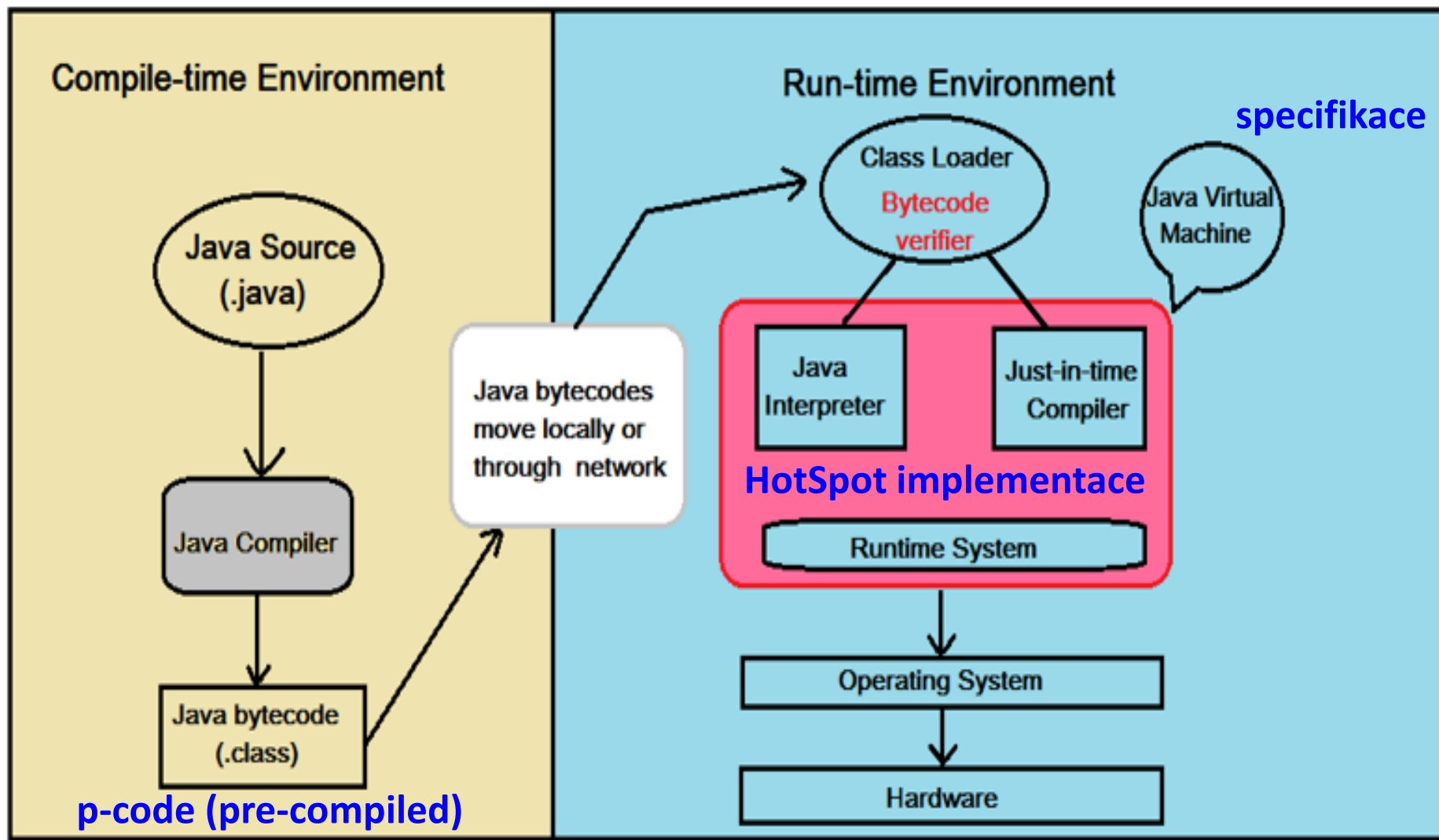
Microsoft .NET: CIL (mezikód) + CLR (runtime)



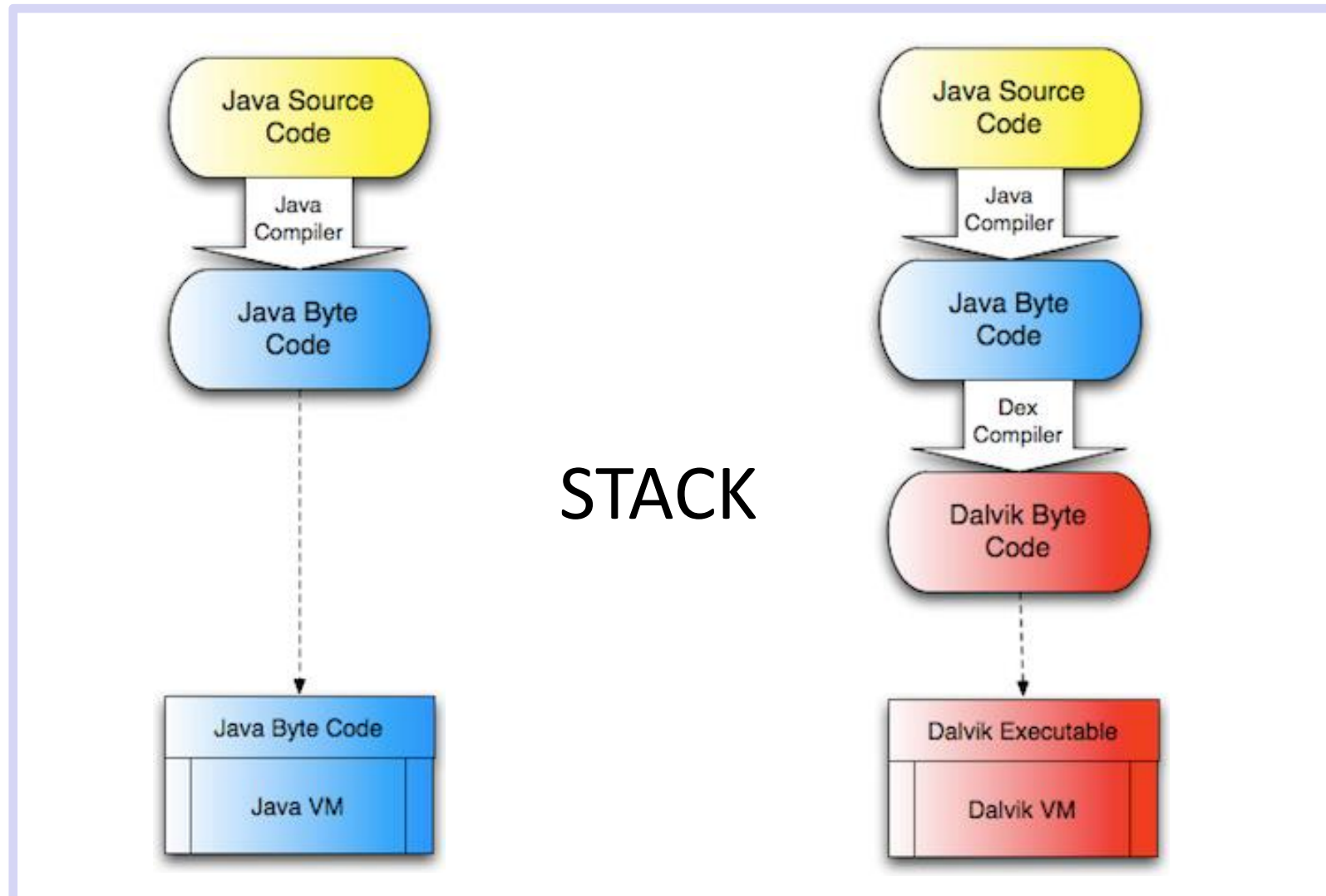
REGISTER

Java

STACK



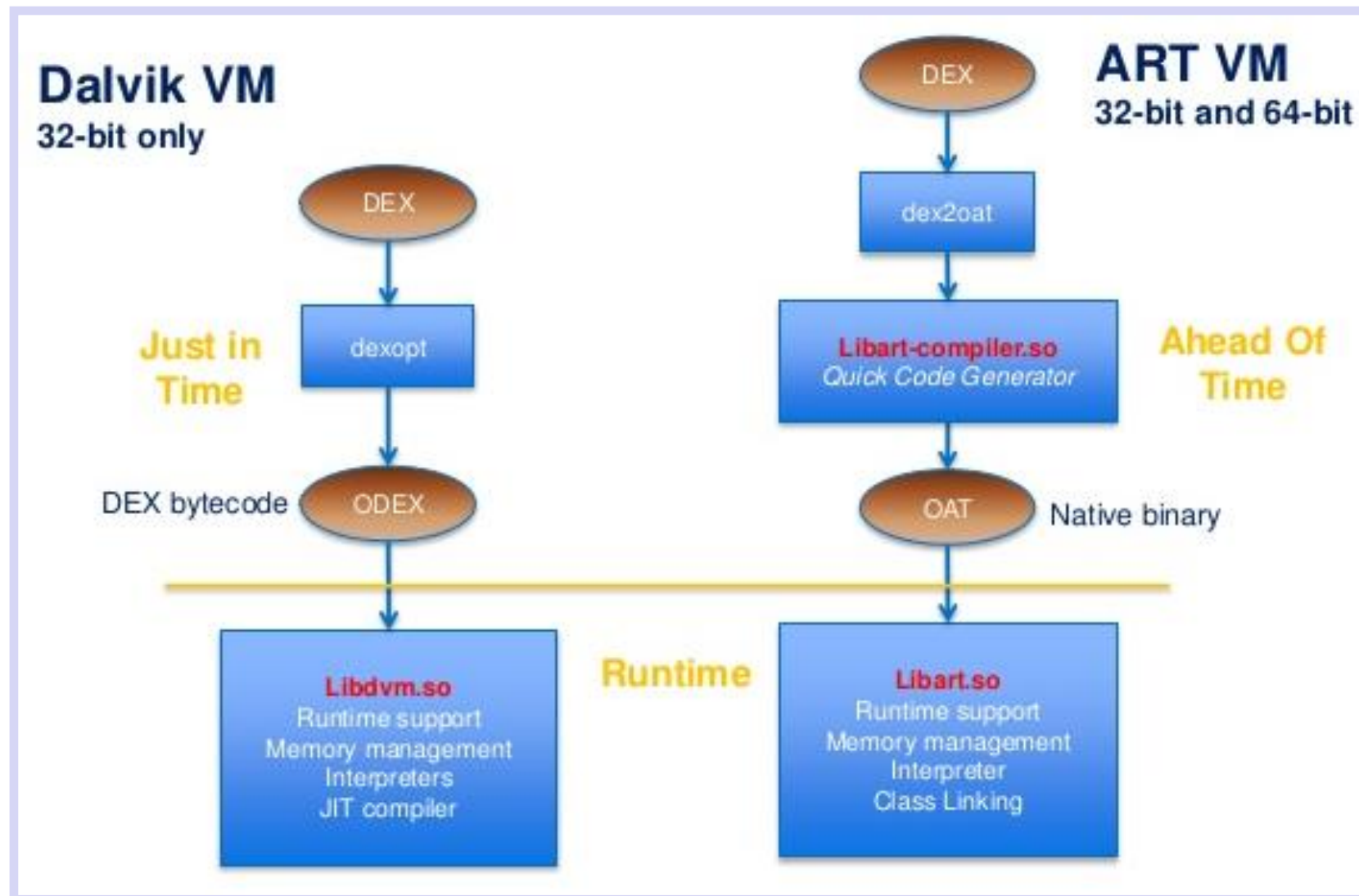
Java VM a Dalvik VM (Android do v.4.4): zásobníkové stroje



Zdroj: <https://markfaction.wordpress.com/2012/07/15/stack-based-vs-register-based-virtual-machine-architecture-and-the-dalvik-vm/>

Dalvik VM a ART (Android RunTime) od v.5.0

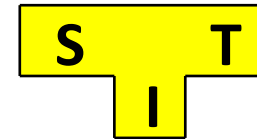
STACK



REGISTER

Bloková reprezentace překladu

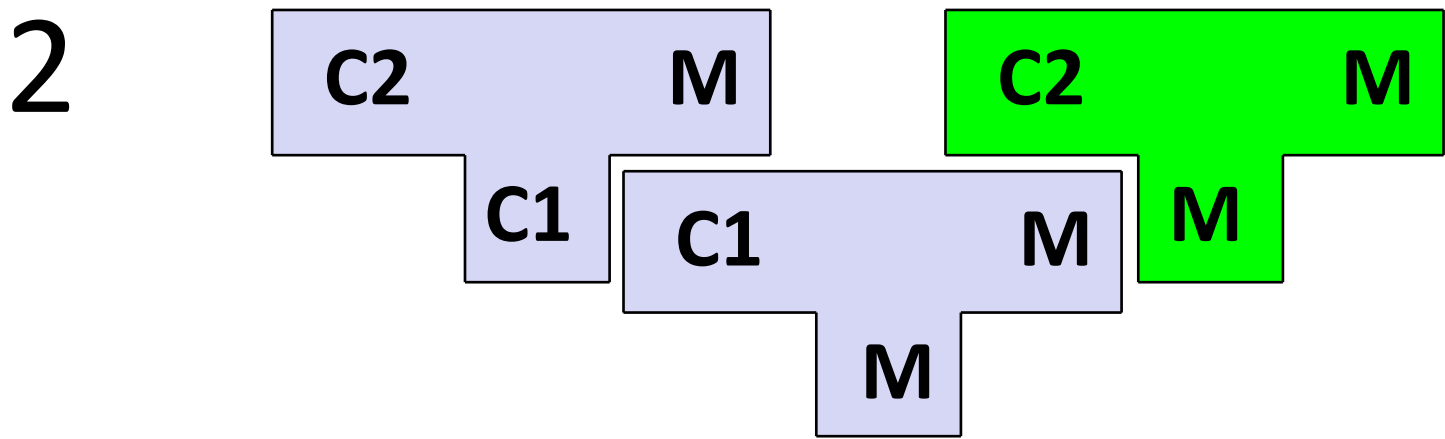
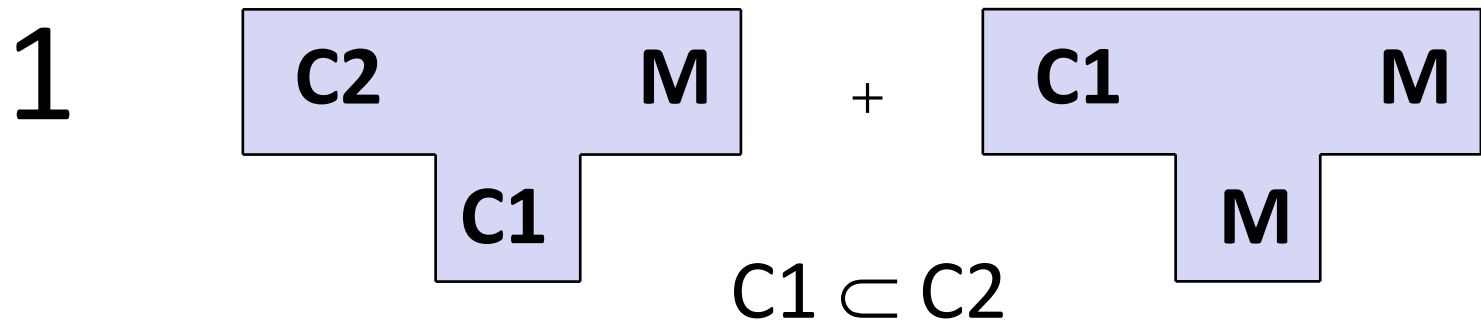
- Pracujeme s T, někdy zvanými též S_IT diagramy:
 - S: zdrojový jazyk (source), front end, **CO PŘEKLÁDÁME**
 - Naše gramatiky
 - T: cílový jazyk (target), back end, **DO ČEHO PŘEKLÁDÁME**
 - Zadaný mezikód (např. TAC ve formě čtveřic)
 - I: implementační jazyk (implementation), **ČÍM PŘEKLÁDÁME**
 - C (lex a yacc jsou nadstavbové nástroje)
- U překladače je S vždy různé od T, ale S může být shodné s I a I může být shodné s T
 - C++_C = první překladač C++
 - Zdrojový kód v C++ je přeložen do C, aby mohlo být dále zpracován (spuštěn) standardním kompilátorem jazyka C



Jak napsat překladač

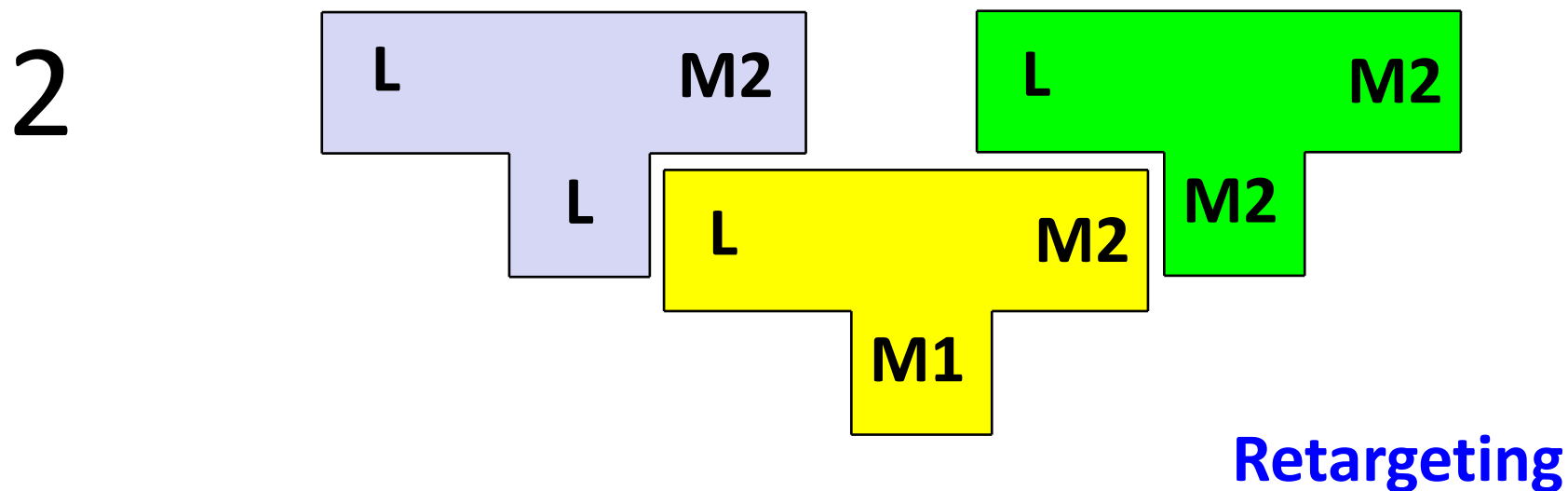
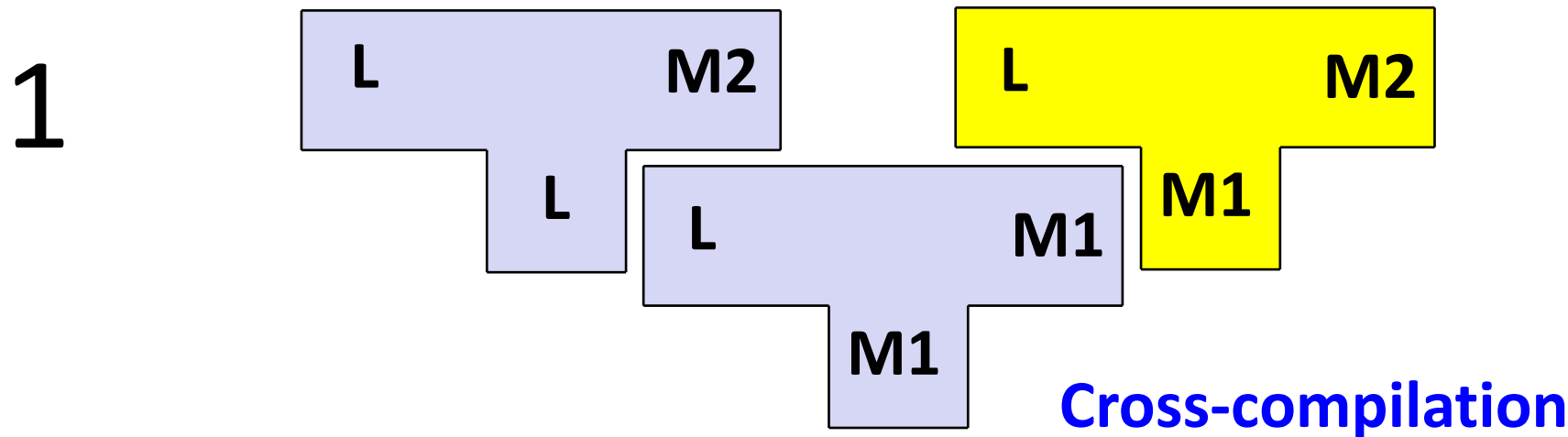
- Kompletně ve strojovém kódu
- V nižším jazyce, k němuž je k dispozici nativní překladač
 - $S_T T$: nativní překladač, např. $C_{x86} x86$
- V tomtéž jazyce (bootstrapping)
 - $L_S T$: bootstrapping, např. $Java_{Java} JVM$, kde $S < L$

Vývoj překladače (nového) jazyka C2 pro M pomocí (stávajícího) jazyka C1



Bootstrapping

Vývoj překladače jazyka L pro platformu M2 na platformě M1



Oblasti rozvoje překladačů

- Nové typy zdrojových jazyků i cílových procesorů
- Virtuální stroje a jejich vlastnosti (JIT, AOT, ...)
- Rozvoj kvalitativních vlastností
 - Rychlost překladu
 - Jde obvykle proti udržitelnosti překladače a kvalitě kódu
 - Snížení počtu sestavovaných modulů
 - Snížení počtu průchodů – ideálně vše v jednom
 - Kvalita a rozsah generovaného kódu
 - Práce s chybami
 - Přenositelnost (portability)
 - Udržitelnost (maintainability)

Jakými jazyky se budeme zabývat?

Půjde o **formální jazyky**, strukturně definované například **gramatikou**. Tu musí ovládat tvůrce **vět** v tomto jazyce, tj. programátor. Věty formálních jazyků jsou strojově rozpoznatelné například tzv. **automatem**, což je vlastně jádro každého překladače.

```
block      ->    begin opt_stmts end

opt_stmts  ->    stmt_list
              |    e (také ε nebo prázdné/nulovací pravidlo

stmt_list  ->    stmt_list ; stmt
              |    stmt

stmt       ->    ...
```

Syntaxe a sémantika

- **Syntaxe:** strukturní předpis

`WHILE <expression> DO <command>`

- **Sémantika:** význam, funkce

`"IF <expression > is TRUE
 THEN perform <command> and repeat action,
IF <expression> is FALSE
 THEN go to the next command,
OTHERWISE error"`

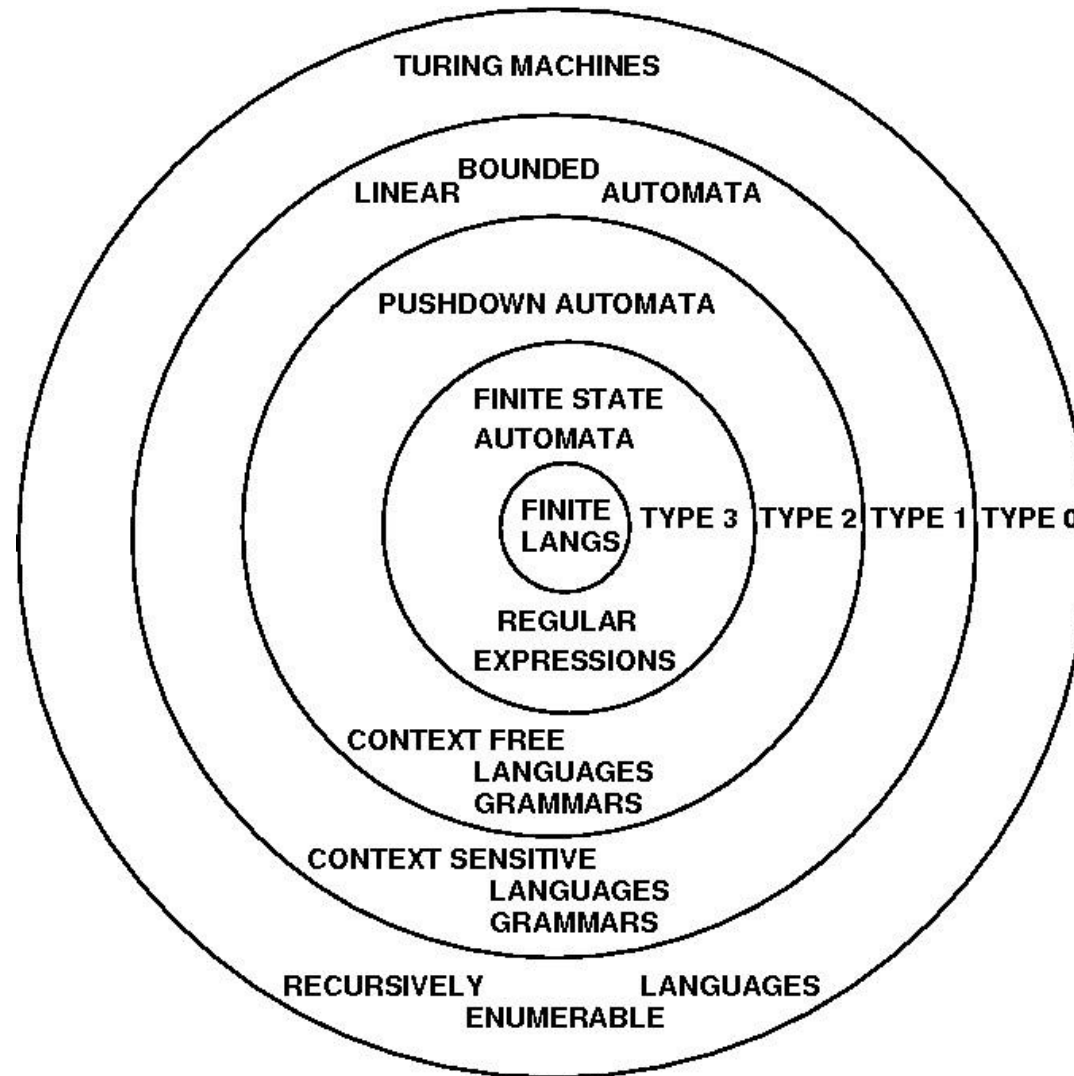
Obě kategorie rozlišujeme proto, že **neumíme** strojově zpracovat gramatiky, které by strukturně reprezentovaly všechny požadované významové (kontextové) aspekty. Běžné gramatiky proto obvykle definujeme jako bezkontextové a kontextově závislé aspekty zajišťujeme jiným způsobem.

Chomského hierarchie jazyků

Grammar	Languages	Automaton	Production rules (constraints)
Type-0	Recursively enumerable	Turing machine	$\alpha \rightarrow \beta$ (no restrictions)
Type-1	Context-sensitive	Linear-bounded non-deterministic Turing machine	$\alpha A \beta \rightarrow \alpha \gamma \beta$
Type-2	Context-free	Non-deterministic pushdown automaton	$A \rightarrow \gamma$
Type-3	Regular	Finite state automaton	$A \rightarrow a$ and $A \rightarrow aB$

$$T3 \subset T2 \subset T1 \subset T0$$

Klasifikace jazyků, gramatik a automatů



Zdroj: http://www.cs.rochester.edu/~brown/173/lectures/flat/formal_lang/Automata5lec.html

Specifikace programovacích jazyků

doc. Ing. Miroslav Beneš, Ph.D.
katedra informatiky FEI VŠB-TUO

A-1007 / 597 324 213

<http://www.cs.vsb.cz/benes>

Miroslav.Benes@vsb.cz



Omezení bezkontextových gramatik

Operátory, známé z oblasti regulárních výrazů, např.

* | + ? ^ []

zde mají význam standardních ASCII znaků. Jejich syntaxi/význam musí definovat samotná gramatika.

Příklad 1

$E \rightarrow \text{int} \mid E \text{ Op } E \mid (E)$

$\text{Op} \rightarrow + \mid - \mid * \mid / \mid$

terminál: zelená barva

Příklad 2

$S \rightarrow a^*b$ $\xrightarrow{\text{realizace}}$

$S \rightarrow Ab$

$A \rightarrow Aa \mid \epsilon$

Příklady: rozhodněte o **regularitě** gramatik

G1:

$$\begin{aligned} S &\rightarrow aX \mid bY \\ X &\rightarrow x \mid xX \\ Y &\rightarrow y \mid yY \end{aligned}$$

G2:

$$\begin{aligned} S &\rightarrow aX \mid bY \\ X &\rightarrow x \mid xX \\ Y &\rightarrow y \mid Yy \end{aligned}$$

G3:

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow a \mid aA \\ B &\rightarrow b \mid bB \end{aligned}$$

G4:

$$S \rightarrow xSy \mid xY$$

Priorita a asociativita

- Priorita: různé úrovně operátorů

$$7 / 4 + 2$$

$$\rightarrow (7 / 4) + 2 = 11/4$$

dělení před sčítáním

$$\rightarrow 7 / (4 + 2) = 7/6$$

sčítání před dělením

- Asociativita: stejné úrovně operátorů

$$7 - 4 + 2$$

$$\rightarrow (7 - 4) + 2 = 5$$

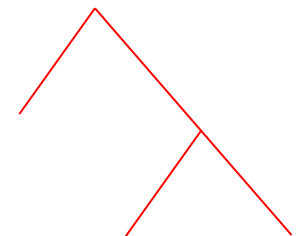
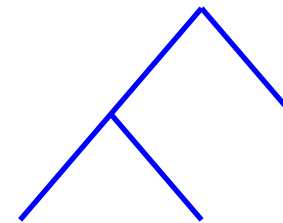
levá

$$\rightarrow 7 (- 4 (+ 2)) = 1$$

pravá

Levá asociativita: levá rekurze

Pravá asociativita: pravá rekurze



Priorita a asociativita operátorů jazyka C

Operator	Description	Associativity
() [] . -> ++ --	Parentheses or function call Brackets or array subscript Dot or Member selection operator Arrow operator Postfix increment/decrement	left to right
++ -- + - ! ~ (type) * & sizeof	Prefix increment/decrement Unary plus and minus not operator and bitwise complement type cast Indirection or dereference operator Address of operator Determine size in bytes	right to left
* / %	Multiplication, division and modulus	left to right
+ -	Addition and subtraction	left to right
<< >>	Bitwise left shift and right shift	left to right
< <= > >=	relational less than/less than equal to relational greater than/greater than or equal to	left to right
== !=	Relational equal to and not equal to	left to right
&	Bitwise AND	left to right
^	Bitwise exclusive OR	left to right
	Bitwise inclusive OR	left to right
&&	Logical AND	left to right
	Logical OR	left to right
? :	Ternary operator	right to left
= += -= *= /= %= &= ^= = <<= >>=	Assignment operator Addition/subtraction assignment Multiplication/division assignment Modulus and bitwise assignment Bitwise exclusive/inclusive OR assignment	right to left
,	Comma operator	left to right

Poznámky:

1. Nejdříve priorita, pak asociativita
2. Stejná priorita = stejná asociativita
3. Závorky vždy vše zpřehlední

Příklad:

100 + 200 / 10 - 10 * 50

100 + 200 / 10 - 10 * 50 **priorita**

100 + 20 - 500 **asociativita**
(priorita zleva)

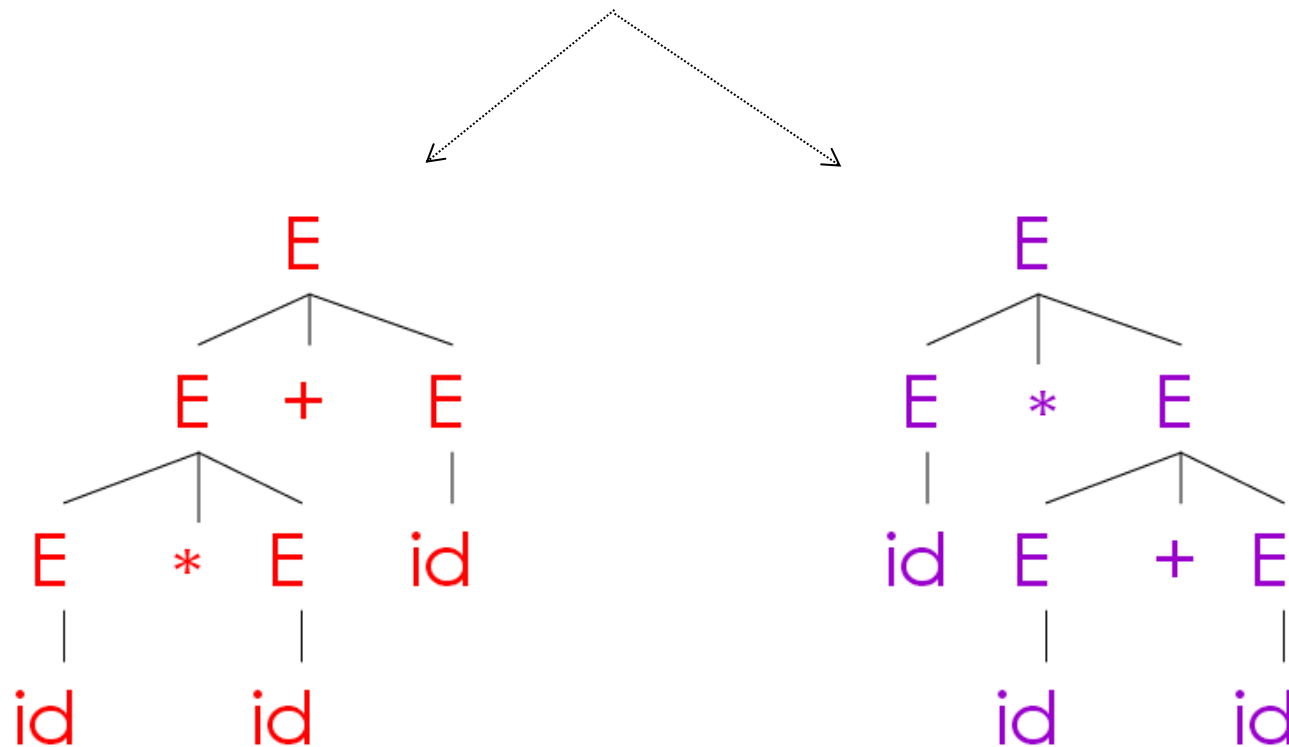
Výsledek: -380

Víceznačnost gramatiky pro výrazy

$E \rightarrow E + E \mid E * E \mid (E) \mid id$

Vstup:

id * id + id



Odstranění víceznačnosti

I. Změna gramatiky

$E \rightarrow E - E \mid E / E \mid (E) \mid \text{id}$

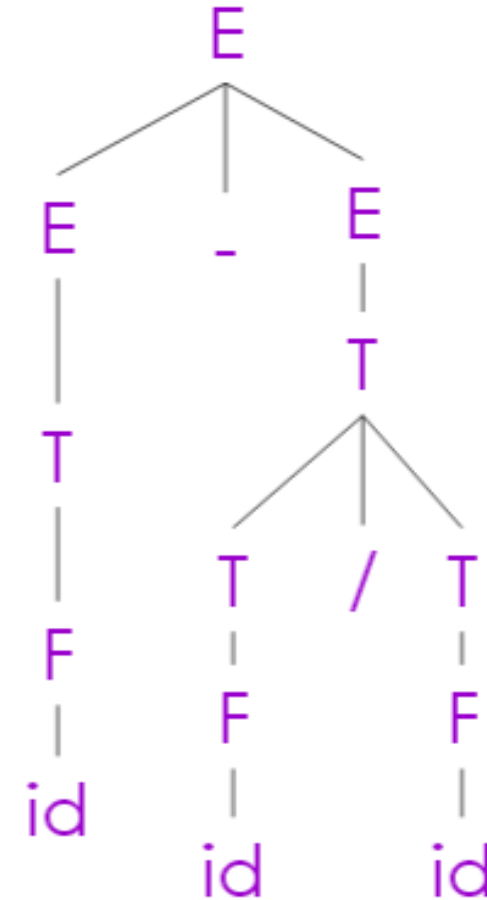


$E \rightarrow E - E \mid T$

$T \rightarrow T / T \mid F$

$F \rightarrow \text{id} \mid (E)$

Příklad: $\text{id} - \text{id} / \text{id}$



II. Zavedení dodatečných pravidel

– *yacc/bison*

Nejednoznačnost (ambiguity) - dangling *else*

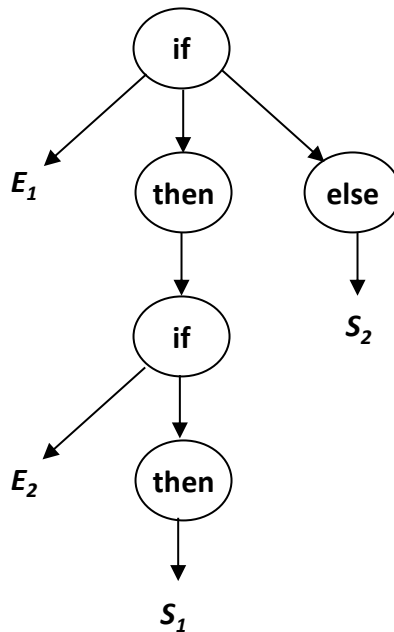
Stmt --> if Expr then Stmt (1)

| if Expr then Stmt else Stmt (2)

| ... ostatní příkazy jazyka ...

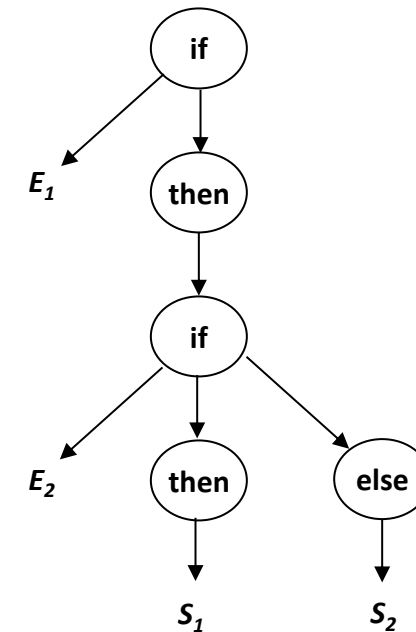
← Gramatika

if Expr1 then if Expr2 then Stmt1 else Stmt2 ← Jazyk



Pravidlo 2, pak pravidlo 1

Oba stromy jsou
syntakticky správné.
Default řešení: *else* se
páruje s nejbližším *then*



Pravidlo 1, pak pravidlo 2

Příklad

- Poslání balíčku uvnitř USA stojí 5 USD
- Výjimkou je stát Hawaii, kam se platí 10 USD
- Poslání balíčku mimo USA stojí 20 USD

```
poštovné = 5;  
if ( země == "USA")  
    if ( stát == "Hawaii" )  
        poštovné = 10.00;  
else  
    poštovné = 20.00;
```

Řešení č. 2 (LCI web, E4/19):
Změna gramatiky, rozlišující spárované a nespárované *else*:

```
stmt ->  
    matched_stmt | open_stmt  
  
matched_stmt ->  
    if expr then matched_stmt else matched_stmt  
    | other  
  
open_stmt ->  
    if expr then stmt  
    | if expr then matched_stmt else open_stmt
```

Řešení č. 1 :

Rozšíření gramatiky o identifikátory rozsahu platnosti, např.

if - then - fi, if - then else - fi,
if {then}, if {then} else,

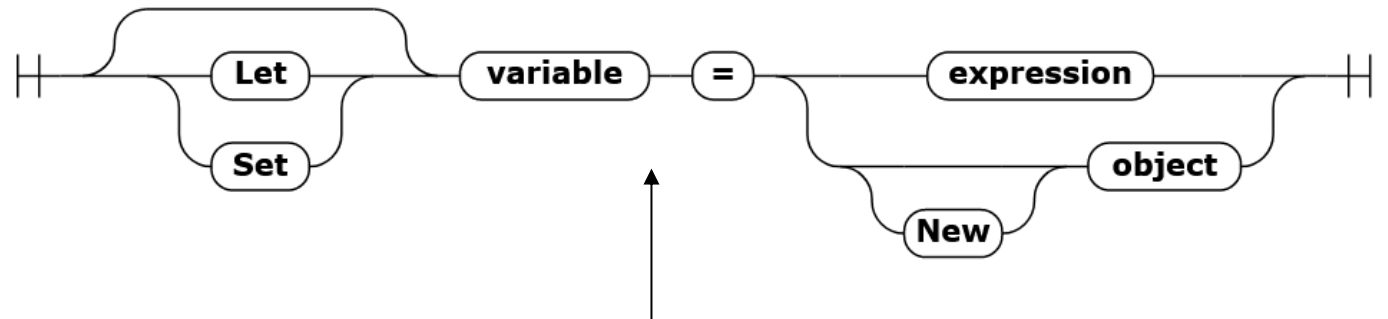
C++: then je nahrazeno { ... } :

```
if ( země == "USA")    {  
    if ( stát == "Hawaii" )  
        poštovné = 10.00; }  
else ...
```

Opakovací příklady ke gramatikám, shrnutí



- Jeden jazyk může být generován více gramatikami
- Při konstrukci překladačů nás zajímají:
 - Regulární gramatiky
 - Bezkontextové gramatiky
- Jazyky charakterizujeme:
 - Výčtem
 - Slovně
 - Množinově
 - Formalizovaným zápisem (RE, Backusova–Naurova forma[BNF])
 - Gramatikou
 - Automatem



Reprezentace jazyků slovně a množinově

- Jazyk, tvořený všemi řetězci, obsahujícími libovolný počet znaků x .

$$\{x^n \mid n > 0\}$$

- Jazyk, tvořený všemi řetězci, obsahujícími nejméně jeden znak x , následovaný vždy stejným počtem znaků y

$$\{x^n y^n \mid n > 0\}$$

- Jazyk, tvořený všemi řetězci, obsahujícími nejméně jeden znak x , následovaný vždy nejméně jedním řetězcem y , přičemž počet znaků x a y nemusí být shodný

$$\{x^n y^m \mid n, m > 0\}$$

Reprezentace jazyků regulárními výrazy (RE)

- Popište předchozí jazyk pomocí regulárního výrazu (RE)

x^+y^+

nebo

xx^*yy^*

- Napište RE generující žádný nebo více znaků x nebo žádný nebo více znaků y

$x^* \mid y^*$

- Napište RE generující žádný nebo více znaků a nebo b

$(a \mid b)^*$

- Napište RE generující jeden znak a a žádný nebo více znaků b

$a \mid b^*$

Další příklady na jazyky a gramatiky

- Napište několik řetězců, generovaných RE $(aab/ab)^*$
 $empty(\varepsilon), \quad ababab, \quad aababaab$
- Je RE $(abc)^+$ ekvivalentní výrazu $(abc)(abc)^*$?
 ANO
- Vyjádřete vzory pro identifikátory a čísla s pevnou desetinnou tečkou jako RE
 $L(L \mid D)^* \quad (D^*D.D^*) \mid (D^*.DD^*)$
- Lze definovat jazyk, párující oddělovače, pomocí RE?
Protože tato konstrukce používá čítač, nelze k její definici použít RE, ale nejméně CFG, například s těmito pravidly:

$$S \rightarrow xSy \mid xy$$

Jazyk generovaný regulárním výrazem (RE)

Který z následujících jazyků nad slovníkem $\{0,1\}$ je reprezentován tímto regulárním výrazem:

$$(0+1)^*0(0+1)^*0(0+1)^*$$

Odpověď: je to množina všech řetězců:

- a) Obsahujících podřetězec 00
- b) Obsahujících nejvýše dvě nuly
- c) Obsahujících nejméně dvě nuly
- d) Začínajících nebo končících buď 0 nebo 1

Jazyk generovaný regulární gramatikou (RG)

Který z jazyků a) – d) je generován následující gramatikou:

$$S \rightarrow aS \mid bS \mid \varepsilon$$

Odpověď:

a) $\{a^n b^m \mid n, m \geq 0\}$

b) $\{w \in \{a, b\}^* \mid w \text{ má stejný počet znaků } a \text{ i } b\}$

c) $\{a^n \mid n \geq 0\} \cup \{b^n \mid n \geq 0\} \cup \{a^n b^n \mid n \geq 0\}$

d) $\{a, b\}^*$

Slovně charakterizujte následující RE

$0(0|1)^*0$

Řetězce začínající a končící 0

$((\varepsilon|0)1^*)^*$

Libovolné řetězce

$(0|1)^*0(0|1)(0|1)$

Řetězce, obsahující 0 na třetí pozici zprava

$0^*10^*10^*10^*$

Řetězce, obsahující právě tři jedničky

Popis jazyka pomocí RE a gramatiky

Jazyk všech řetězců nad abecedou $\{0,1\}$ takový, že každá 0 je okamžitě následována nejméně jednou 1. Nesmí být prázdný.

- RE: $1(011^*)^*$
- Bezkontextová (CFG) gramatika 1

$$S \rightarrow SS|0A|1$$

$$A \rightarrow 1A|1$$

RE a gramatika pro totožný jazyk s možným prázdným vstupem

- RE: $1^*(011^*)^*$
- Bezkontextová gramatika 2

$$S \rightarrow AB$$

$$A \rightarrow \varepsilon \mid 1A$$

$$B \rightarrow \varepsilon \mid CB$$

$$C \rightarrow 01A$$

Popis jazyka pomocí gramatiky

- Jazyk všech párových oddělovačů typu (), [] nebo {}.

$$S \rightarrow (S)S \mid [S]S \mid \{S\}S \mid \varepsilon$$

- Prázdný jazyk všech řetězců, tvořených 0 a 1 takový, že počet 0 odpovídá počtu 1

$$S \rightarrow 0S1S \mid 1S0S \mid \varepsilon$$

- Prázdný jazyk všech řetězců nad abecedou {0,1} takový, že se v něm nemůže vyskytnout podřetězec 011

$$S \rightarrow AB$$

$$A \rightarrow 1A \mid \varepsilon$$

$$B \rightarrow 01B \mid 0B \mid \varepsilon$$

Konečný automat



$$A = (Q, \Sigma, \delta, q_0, F)$$

- Q : (konečná) množina stavů
- Σ : vstupní abeceda
- δ : přechodová funkce

$$\delta_{NKA} : Q \times \Sigma \rightarrow 2^Q$$

$$\delta_{DKA} : Q \times \Sigma \rightarrow Q$$

- q_0 : počáteční stav
- F : množina koncových stavů

Mějme automat $M = (Q, T, d, q_0, F)$, kde:

$Q = \{q_0, q_1\}$

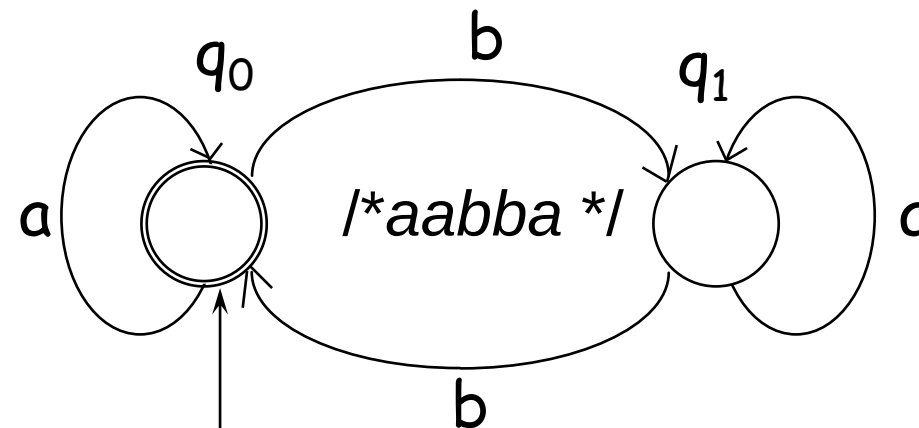
$T = \{a, b\}$

$F = q_0$

d je dáno tabulkou

q	s	$\delta(q, s)$			
q_0	a	q_0	δ	a	b
q_0	b	q_1	q_0	q_0	q_1
q_1	a	q_1	q_1	q_1	q_0
q_1	b	q_0			

Automat lze popsat také stavovým (přechodovým) diagramem:

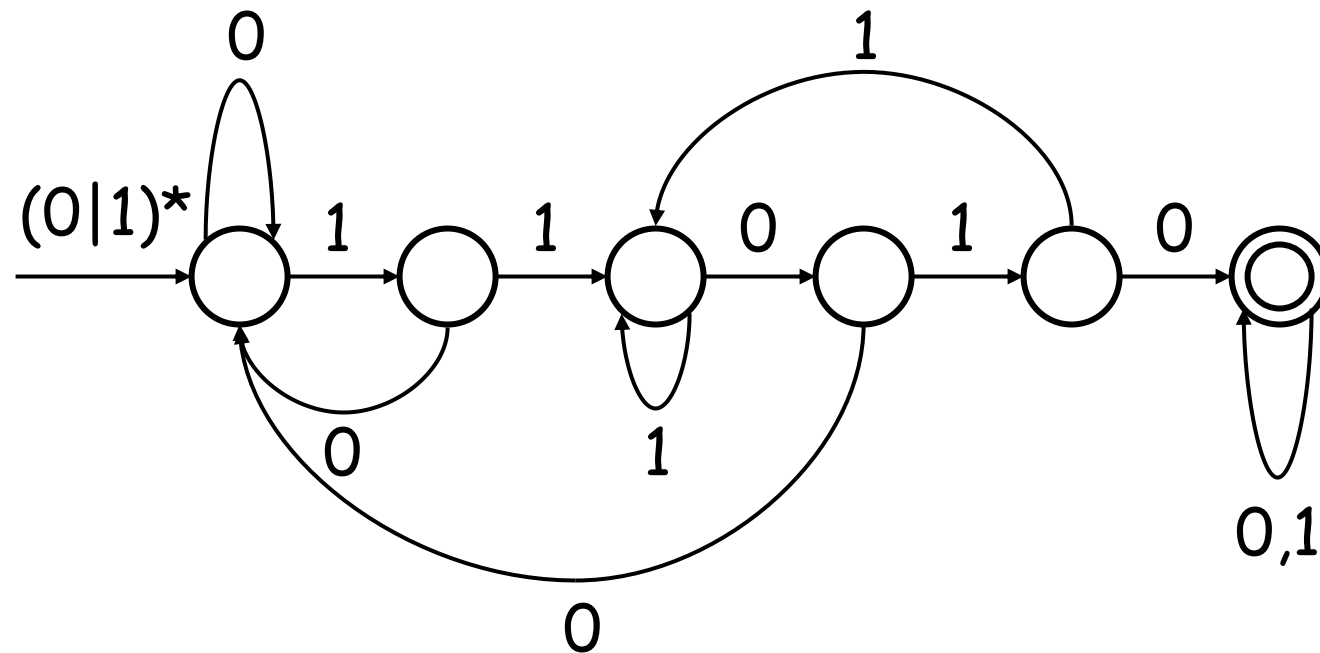


Konečný automat, příklad 1

Navrhněte konečný automat nad abecedou $A=\{0,1\}$, přijímající následující jazyk:

A^* obsahující podřetězec 11010

Řešení:

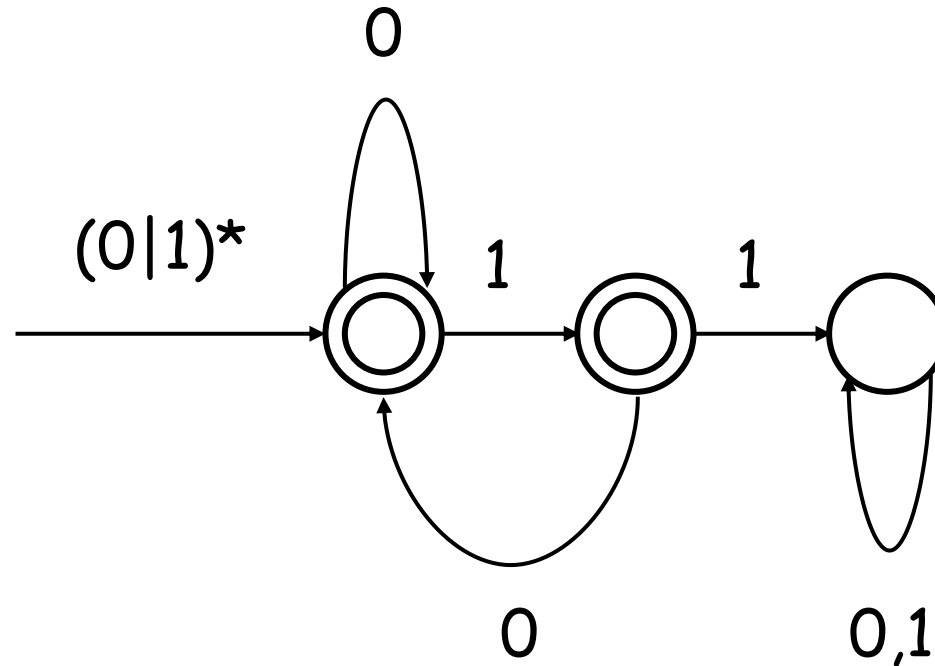


Konečný automat, příklad 2

Navrhněte konečný automat nad abecedou $A=\{0,1\}$, přijímající následující jazyk:

A^* kromě slov, obsahujících podřetězec 11

Řešení:



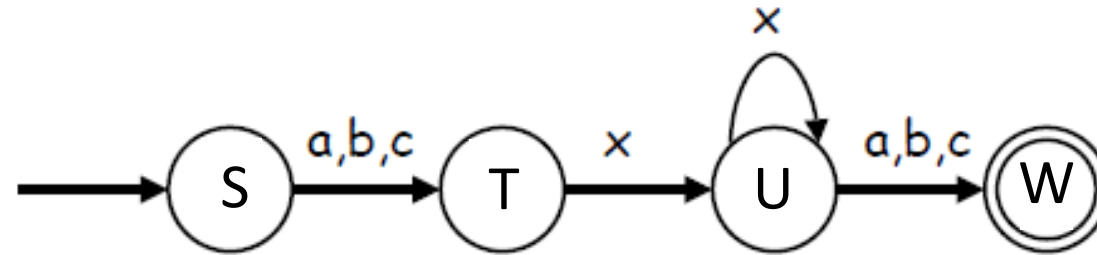
Vztah mezi RE, RG a FA

Pro následující regulární výraz (RE):

$(a|b|c)x^+(a|b|c)$

navrhněte odpovídající konečný automat (FA) a regulární gramatiku (RG).

- **FA:**



- **RG:**

$S \rightarrow aT \mid bT \mid cT$

$T \rightarrow xU$

$U \rightarrow xU \mid aW \mid bW \mid cW$

$W \rightarrow \text{acc.}$

Úkoly na příští týden

- Seznámit se s přednáškovými poznámkami a souvisejícími materiály
- V souladu s rezervační tabulkou průběžně připravovat a odevzdávat individuální prezentace